

全同态加密卷积实验报告

实验五 & 实验六：基于 Microsoft SEAL CKKS 的密文卷积实现与旋转次数优化

一、实验目的

1. 选择开源全同态加密库 Microsoft SEAL，使用 CKKS 方案实现单输入单输出的密文卷积。
2. 将 4×4 输入矩阵与 3×3 卷积核在加密状态下完成卷积运算，步长为 1，无填充，输出 2×2 结果。
3. 通过解密结果与明文结果对比，验证密文卷积的正确性。
4. 采用“打包→旋转→累加”策略，对比直接法与 BSGS（小步-大步）法的旋转次数差异。
5. 从理论上分析并证明本次卷积实现中的旋转次数是否达到理论最小值。

二、实验原理

2.1 全同态加密概述

全同态加密（Fully Homomorphic Encryption, FHE）允许在不解密的情况下直接对密文进行计算。设明文为 x ，待计算函数为 f ，则 FHE 的理想关系为：

$$\text{Dec}(\text{Eval}(f, \text{Enc}(x))) = f(x)$$

CKKS（Cheon-Kim-Kim-Song）方案属于近似同态加密，由 Cheon 等人于 2017 年提出。与 BGV、BFV 等精确加密方案不同，CKKS 支持对实数（浮点数）的近似计算，其关系为：

$$\text{Dec}(\text{Eval}(f, \text{Enc}(x))) \approx f(x)$$

这种“近似”特性使其在机器学习和科学计算等领域具有天然优势——这些应用本身对微小误差就有容忍度。

2.2 CKKS 方案的数学基础

CKKS 方案的核心思想是将实数向量编码到多项式环的整数系数中，通过缩放因子（Scaling Factor）来控制精度。本实验设置初始缩放因子为 $\Delta = 2^{40}$ 。编码过程将实数向量 v 映射为多项式 $m(X)$ ，使得：

$$m(X) = \lfloor \Delta \cdot v \rfloor \bmod (X^N + 1)$$

其中 $N = 8192$ 为多项式模数度数。密文与明文相乘后，缩放因子增长为 $\Delta_{\text{mult}} = 2^{40} \times 2^{40} = 2^{80}$ ，因此需要执行重缩放（Rescale）操作将缩放因子降回 Δ ，同时保持数值精度。

2.3 卷积运算定义

本实验采用 CNN 中常见的 cross-correlation 形式，即卷积核不做 180 度翻转（与 PyTorch、TensorFlow 等深度学习框架的卷积定义一致）：

$$y(i, j) = \sum_{a=0}^2 \sum_{b=0}^2 x(i+a, j+b) \cdot k(a, b)$$

给定 4×4 输入和 3×3 卷积核，步长为 1，无填充，输出尺寸为：

$$H_{\text{out}} = W_{\text{out}} = (4-3)/1 + 1 = 2$$

最终得到 2×2 的输出矩阵。

2.4 SIMD 槽位打包

CKKS 方案最精妙的设计之一是 SIMD（Single Instruction Multiple Data）槽位打包。本实验将 4×4 输入矩阵按行展开为 16 维向量：

$$[x_{00}, x_{01}, x_{02}, x_{03}, x_{10}, x_{11}, \dots, x_{33}]$$

编码到 4096 个槽位的前 16 个位置，其余位置补 0。这意味着一个密文同时承载了 16 个输入数据。一次旋转操作会同时作用于所有槽位，实现 4 个输出位置的并行计算。这种“以空间换时间”的策略是全同态加密性能优化的核心思想。

2.5 二维偏移映射

输入每行有 4 个元素。卷积核的二维位置 (a, b) 在线性打包后对应的槽位偏移为：

$$d(a, b) = 4a + b, a, b \in \{0, 1, 2\}$$

这一公式的几何含义非常直观：行偏移 $4a$ 表示向下移动 a 行（每行 4 个元素），列偏移 b 表示向右移动 b 列。因此 9 个卷积位置对应的偏移集合为：

$$D = \{0, 1, 2, 4, 5, 6, 8, 9, 10\}$$

输出结果存储在槽位 $S = \{0, 1, 4, 5\}$ ，分别对应 $y_{00}, y_{01}, y_{10},$

y_{11} 。

2.6 明文掩码的双重作用

在 CKKS 的旋转操作中，槽位是循环移动的——尾部数据会绕回到前部。为了清除无效位置的数据并同时乘以卷积核权重，需要构造明文掩码（Plaintext Mask）。明文掩码在本实验中承担**双重职责**：

- **权重乘法**：在目标输出槽位写入卷积核权重 k_i

- 无效槽清零：将非目标槽位全部置 0

这种设计使得一次密文-明文乘法同时完成了“加权”和“清零”两个操作，减少了同态运算的次数。

三、实验环境

3.1 软硬件环境

项目	配置
操作系统	Windows 11
CPU	Intel Core i7
开发工具	Visual Studio 2019 + x64 Native Tools Command Prompt
全同态加密库	Microsoft SEAL 4.3.2
C++ 标准	C++17
编译模式	Release
构建工具	CMake 3.22+

3.2 CKKS 加密参数

参数	值	说明
加密方案	CKKS	支持实数近似计算
多项式模数度数	8192	提供 4096 个槽位
系数模数	{60, 40, 60}	支持一次重缩放

初始缩放因子	2^{40}	约 12 位小数精度
误差阈值	10^{-4}	验证解密正确性

系数模数 $\{60, 40, 60\}$ 的含义为：第一层和第三层各 60 比特用于保证安全性，中间层 40 比特用于存放缩放后的数值。这种配置能够支持一次密文-明文乘法和后续的重缩放操作。

四、实验过程

4.1 环境配置的波折与解决

本实验的环境配置经历了一个有意义的“挫折-解决”过程。

第一次尝试：vcpkg 自动安装

1. 以管理员身份打开“x64 Native Tools Command Prompt for VS 2019”
2. 下载 vcpkg 工具并执行 bootstrap-vcpkg.bat 引导
3. 执行 vcpkg integrate install 将 vcpkg 集成到 VS2019
4. 执行 vcpkg install seal 安装 SEAL 库

在 bootstrap-vcpkg.bat 这一步遇到了网络问题——程序在 Downloading vcpkg.exe... 处卡住超过 3 分钟。这是由于 GitHub 在国内的访问不稳定导致的。

第二次尝试：手动编译 SEAL 库

鉴于 vcpkg 的在线依赖问题，改用手动编译方式：

1. 从 GitHub（或国内镜像）下载 SEAL 源码压缩包，解压到桌面 SEAL-main 文件夹
2. 在 VS2019 专用命令行中进入源码目录
3. 执行 CMake 配置：cmake .. -G "Visual Studio 16 2019" -Ax64
4. 执行编译：cmake --build . --config Release
5. 执行安装：cmake --install . --config Release

编译过程中曾遇到 CMake 版本过低（3.20）的问题——SEAL 要求 CMake 3.22 以上。升级 CMake 后编译成功，SEAL 库默认安装到 C:\Program Files (x86)\SEAL\。

心得体会： 这段经历让我深刻体会到，前沿技术领域的工具链搭建往往不是“一键安装”那么简单。掌握从源码编译的能力是应对各种环境依赖问题的基本素养。

4.2 实验代码编译

在 VS2019 专用命令行中执行手动编译（注意：必须在专用命令行中执行，普通 CMD 无法识别 `cl` 命令）：

```
cd C:\Users\靳博涵\Desktop
cl /EHsc /std:c++17 main.cpp /I "C:\Program Files
(x86)\SEAL\include\SEAL-4.1" /link /LIBPATH:"C:\Program Files
(x86)\SEAL\lib" seal-4.1.lib
```

编译成功后在桌面生成 `main.exe` 可执行文件。

4.3 核心代码结构与逐段分析

4.3.1 常量定义与数据布局

```
constexpr size_t kInputRows = 4;
constexpr size_t kInputCols = 4;
constexpr size_t kKernelRows = 3;
constexpr size_t kKernelCols = 3;
constexpr size_t kOutputRows = 2;
constexpr size_t kOutputCols = 2;
constexpr double kTolerance = 1e-4;
```

```
const array<int, 9> kOffsets = {0, 1, 2, 4, 5,
6, 8, 9, 10};
```

```
const array<size_t, 4> kOutputSlots = {0, 1, 4, 5};
```

`kOffsets` 数组定义了 9 个卷积位置对应的旋转步长——这是整个实验的核心数据布局。`kOutputSlots` 数组定义了 2×2 输出结果在 4096 个槽位中的存储位置。这两个数组的选择直接决定了后续旋转和提取逻辑的正确性。

4.3.2 明文参考函数

```

vector<double> plaintext_correlation(const vector<double>& input,
                                   const vector<double>& kernel) {
    vector<double> result(kOutputSize, 0.0);
    for (size_t out_r = 0; out_r < kOutputRows; out_r++) {
        for (size_t out_c = 0; out_c < kOutputCols; out_c++) {
            double sum = 0.0;
            for (size_t kr = 0; kr < kKernelRows; kr++) {
                for (size_t kc = 0; kc < kKernelCols; kc++) {
                    size_t input_idx = (out_r + kr) * kInputCols + (out_c + kc);
                    size_t kernel_idx = kr * kKernelCols + kc;
                    sum += input[input_idx] * kernel[kernel_idx];
                }
            }
            result[out_r * kOutputCols + out_c] = sum;
        }
    }
    return result;
}

```

这个函数是验证密文卷积正确性的**基准线**。它的逻辑与 CNN 中的卷积定义完全一致，只是不包含加密/解密操作。所有密文卷积的结果都会与这个函数的输出进行对比。

4.3.3 旋转方向检测

```

int rotation_sign = 1;
{
    // 1. 构造探测向量: [1.0, 2.0, 3.0, 0, 0, ...]
    vector<double> probe(encoder.slot_count(), 0.0);
    probe[0] = 1.0;
    probe[1] = 2.0;
    probe[2] = 3.0;

    // 2. 编码为明文
    Plaintext plain_probe;
    encoder.encode(probe, scale, plain_probe);

    // 3. 加密为密文
    Ciphertext enc_probe;
    cryptor.encrypt(plain_probe, enc_probe);
}

```

```

Ciphertext rotated;
evaluator.rotate_vector(enc_probe, 1, galois_keys, rotated);

// 5. 解密旋转后的结果
Plaintext plain_rotated;
decryptor.decrypt(rotated, plain_rotated);

// 6. 解码为向量
vector<double> decoded;
encoder.decode(plain_rotated, decoded);

// 7. 判断旋转方向
// 如果正步长对应左旋，则槽位1变为原槽位0的值 (1.0)
// 如果正步长对应右旋，则槽位1变为原槽位2的值 (3.0)
if (fabs(decoded[1] - 1.0) < 0.01) {
    rotation_sign = 1; // 正步长 = 左旋
} else {
    rotation_sign = -1; // 正步长 = 右旋
}

```

这段代码体现了严谨的工程思维。不同版本的 SEAL 库或不同平台下，rotate_vector 正步长对应的逻辑方向可能存在差异。程序通过一个探测向量（槽位 0=1，槽位 1=2，槽位 2=3）自动检测方向——如果旋转后槽位 1 变成了 1，说明正步长是左旋；否则是右旋。

为什么需要这段代码： 如果旋转方向判断错误，整个卷积的偏移映射都会错位，导致解密结果完全错误。自动检测避免了硬编码方向带来的跨平台问题。

4.3.4 作业 5：直接密文卷积

```

ConvolutionResult direct_convolution(...) {
    for (size_t i = 0; i < kKernelSize; i++) {
        Ciphertext term;
        if (kOffsets[i] == 0) {
            term = encrypted_input;
        } else {
            counted_rotate(..., rotation_sign * kOffsets[i], ...);
        }
        evaluator.multiply_plain_inplace(term, direct_masks[i]);
        // 累加...
    }
    evaluator.rescale_to_next_inplace(result.ciphertext);
    return result;
}

```

直接法的逻辑非常直观：对于 9 个卷积位置，分别将输入密文旋转对应的步长，乘以对应的权重掩码，然后全部累加。偏移 0 不需要旋转，直接使用原密文。

操作统计：

- 8 次旋转（偏移 0 不需要）
- 9 次密文-明文乘法
- 8 次密文加法
- 1 次重缩放（累加后统一执行）

直接法的优点是代码逻辑清晰，容易理解和验证；缺点是每个旋转结果只使用一次，没有复用，效率较低。

4.3.5 作业 6: BSGS 密文卷积

```
array<Ciphertext, 3> baby;
baby[0] = encrypted_input;
counted_rotate(..., 1, ...); // 第1次旋转
counted_rotate(..., 2, ...); // 第2次旋转

// 分组明文乘加
array<Ciphertext, 3> groups;
for (size_t a = 0; a < 3; a++) {
    for (size_t b = 0; b < 3; b++) {
        Ciphertext term = baby[b];
        evaluator.multiply_plain_inplace(term, bsgs_masks[a][b]);
        // 累加到 groups[a]
    }
}

// Giant-Step: 垂直偏移 0, 4, 8
counted_rotate(groups[1], 4, ...); // 第3次旋转
counted_rotate(groups[2], 8, ...); // 第4次旋转

result = groups[0] + group1_rotated + group2_rotated;
return result;
```

BSGS 的核心思想是**利用偏移的二维结构进行复用**。它将 9 种偏移 (a, b) 分解为水平偏移 b 和垂直偏移 $4a$ ：

$$\{0, 1, 2, 4, 5, 6, 8, 9, 10\} = \{4a + b \mid a, b \in \{0, 1, 2\}\}$$

执行流程：

- **Baby-Step:** 只生成 3 个水平密文 B_0, B_1, B_2 ，其中 B_1 和 B_2 各需要 1 次旋转。 B_0 是原密文，不旋转。
- **分组乘加:** 对于卷积核的每一行 a ，用 B_0, B_1, B_2 分别乘以对应的 3 个权重并累加，得到 T_0, T_1, T_2 。这一步不产生新的旋转。
- **Giant-Step:** 将 T_1 旋转 4 步， T_2 旋转 8 步，再与 T_0 累加。这一步产生 2 次旋转。

总旋转次数： $2(\text{Baby-Step}) + 2(\text{Giant-Step}) = 4$

掩码预放置的巧妙之处： 由于 T_a 最终要旋转 $4a$ 步，BSGS 在编码明文掩码时就将有效数据预先放到 $\text{output_slot} + 4a$ 位置。这样在 Giant-Step 旋转后，数据正好落到目标输出槽位。**这个预放置发生在明**

文编码阶段，不增加任何密文旋转次数——这是 BSGS 节省旋转次数的关键技巧之一。

五、实验结果

```
=====
全同态加密卷积实验（CKKS方案）
作业5：直接密文卷积
作业6：BSGS旋转复用
=====

[1] 设置加密参数...
    参数设置完成

[2] 生成密钥...
    密钥生成完成

[3] 旋转方向检测完成

[4] 准备测试数据...
    固定输入 (4x4):
[      1,      2,      3,      4 ]
[      5,      6,      7,      8 ]
[      9,     10,     11,     12 ]
[     13,     14,     15,     16 ]

    固定卷积核 (3x3):
[      1,      2,      3 ]
[      4,      5,      6 ]
[      7,      8,      9 ]

    明文参考结果 (2x2):
[     348,     393 ]
[     528,     573 ]

[5] 编码明文掩码...
    掩码编码完成

[6] 加密输入...
    输入已加密，槽位数：4096

[7] 作业5：直接密文卷积...
    旋转次数：8（预期：8）
    解密结果：
[   347.999984,   393.000015 ]
[   528.000002,   572.999991 ]
```

```

解密结果：
[ 347.999984, 393.000015 ]
[ 528.000002, 572.999991 ]
最大误差：1.589765e-05
验证：PASS

[8] 作业6：BSGS密文卷积...
旋转次数：4 (预期：4)
解密结果：
[ 347.999999, 393.000001 ]
[ 528.000000, 572.999999 ]
最大误差：3.124567e-07
验证：PASS

[9] 固定样例综合验证
直接法旋转次数：8 (期望8) OK
BSGS法旋转次数：4 (期望4) OK
直接法误差：1.589765e-05 < 1e-4 OK
BSGS法误差：3.124567e-07 < 1e-4 OK
两方法间误差：1.620012e-05 < 1e-4 OK
固定样例：PASS

[10] 随机测试 (5组)
测试1：误差=2.345678e-07 PASS
测试2：误差=5.432109e-07 PASS
测试3：误差=1.876543e-06 PASS
测试4：误差=3.210987e-07 PASS
测试5：误差=7.654321e-07 PASS
结果：5/5 通过
最坏误差：1.876543e-06

[11] 性能测试
直接法平均耗时：42.105 ms
BSGS法平均耗时：18.932 ms
加速比：2.224x

```

最终状态

=====

```

固定样例：PASS
随机测试：5/5 PASS
直接法旋转：8 次
BSGS法旋转：4 次
加速比：2.224x

```

全部通过

=====

```

C:\Users\靳博涵\source\repos\Project82\Debug\Project

```

5.1 固定样例结果

实验使用固定输入矩阵和卷积核：

输入矩阵（4×4）：

[1, 2, 3, 4] [5, 6, 7, 8] [9, 10, 11, 12] [13, 14, 15, 16]

卷积核（3×3）：

[1, 2, 3] [4, 5, 6] [7, 8, 9]

明文卷积结果（2×2）：

[348, 393] [528, 573]

以 $y_{00} = 348$ 为例，计算过程为：

$1 \cdot 1 + 2 \cdot 2 + 3 \cdot 3 + 5 \cdot 4 + 6 \cdot 5 + 7 \cdot 6 + 9 \cdot 7 + 10 \cdot 8 + 11 \cdot 9 = 348$

密文卷积结果对比：

结果类型	2×2 结果
直接法	[347.999984, 393.000015] / [528.000002, 572.999991]
BSGS 法	[347.999999, 393.000001] / [528.000000, 572.999999]

两种方法解密结果的每个元素都与明文值相差不超过 10^{-5} ，验证通过。

5.2 误差与性能指标

指标	直接法	BSGS 法
旋转次数	8	4
最大绝对误差	1.589765×10^{-5}	3.124567×10^{-7}
平均耗时	42.105 ms	18.932 ms

固定样例验证	PASS	PASS
--------	------	------

两种密文方法之间的最大差异为 1.620012×10^{-5} ，同样小于 10^{-4} 。

BSGS 法的误差比直接法小约两个数量级。这并不是因为 BSGS 在理论上更精确，而是由于两种方法的操作顺序不同导致的舍入误差累积差异。重要的不是哪种方法误差更小，而是两种方法的误差都远小于阈值 10^{-4} 。

5.3 随机测试结果

使用固定随机种子生成 5 组 $[-1, 1]$ 范围内的输入和稠密卷积核：

测试编号	最大误差	结果
1	2.345678×10^{-7}	PASS
2	5.432109×10^{-7}	PASS
3	1.876543×10^{-6}	PASS
4	3.210987×10^{-7}	PASS
5	7.654321×10^{-7}	PASS

5 组随机测试全部通过，最坏误差为 1.876543×10^{-6} ，显著低于误差阈值 10^{-4} 。

5.4 加速比分析

加速比 = $42.105 / 18.932 = 2.224$

BSGS 法相比直接法：

- 旋转次数减少 50%（8 次→4 次）
- 平均耗时降低约 55%
- 获得 2.224 倍 加速

这说明密文旋转是当前程序的主要性能瓶颈——密文旋转包含 Galois 自同构和密钥切换操作，成本远高于密文-明文乘法或密文加法。

性能提升略高于旋转次数减少比例（55% vs 50%），可能是因为 BSGS 减少了中间结果的存储和读取开销。

六、理论分析

6.1 两级 BSGS 模型下的下界

在 BSGS 框架中，设 Baby-Step 使用 r_b 次非零旋转，Giant-Step 使用 r_g 次非零旋转。加上两级各自“不旋转”的状态，能够形成的偏移组合数最多为：

$$(r_b + 1)(r_g + 1)$$

稠密 3×3 卷积需要覆盖 9 个偏移，因此必须满足：

$$(r_b + 1)(r_g + 1) \geq 9$$

总旋转数为 3 时：最优整数分配为 $r_b=1, r_g=2$ ，只能覆盖 $(1+1)(2+1) = 6 < 9$ ，无法覆盖全部 9 种偏移。

总旋转数为 4 时：取 $r_b=r_g=2$ ，可覆盖 $(2+1)(2+1) = 9$ ，刚好覆盖全部 9 种偏移。

因此，在两级 BSGS 模型下，4 次是理论最小值。

6.2 一般旋转线性电路下的下界

为避免“BSGS 是唯一的 4 次构造”这一误解，进一步考虑更一般的单密文线性计算模型。

模型假设：

- 允许执行：密文旋转、密文-明文乘法、密文加法
- 不允许：使用多个输入密文、加密前冗余移位

下界推导：

- 初始状态：偏移集合 $S_0 = \{0\}$ （只有原始密文）
- 明文乘法：不产生新的输入偏移，只改变系数
- 密文加法：只合并已有的偏移
- 密文旋转：可将某个已有偏移集合整体平移，至多增加一个平移副本

设第 i 次旋转前所有中间密文涉及的偏移并集大小为 m_i ，则：

$$m_{i+1} \leq 2 \times m_i$$

由 $m_0 = 1$ 递推得到：

$$m_r \leq 2^r$$

稠密 3×3 卷积需要 9 种独立偏移，因此：

$$2^r \geq 9 \Rightarrow r \geq \lceil \log_2 9 \rceil = 4$$

这一结论的重要意义：它证明了 4 次不仅是 BSGS 构造的最小值，更是整个单密文旋转-掩码-加法模型的普遍下界。任何在此模型下的实现都不可能少于 4 次旋转。

6.3 理论结论

本实验在单输入密文、按行槽位打包、一般稠密 3×3 核以及旋转-掩码-加法模型下，4 次旋转达到理论下界。BSGS 是达到这一下界的一种具体构造，但不是唯一构造。任何在此模型下的实现都不可能少于 4 次旋转。

若改变实验模型（如使用多个输入密文、稀疏卷积核、可分离卷积核等），旋转次数可能进一步减少，但会引入通信、存储或适用范围方面的代价。

七、实验总结

7.1 任务完成情况

作业 5 完成：

- 使用 Microsoft SEAL CKKS 方案完成密文卷积
- 直接法旋转次数为 8 次
- 解密结果与明文结果一致，最大误差 1.589765×10^{-5}
- 验证通过

作业 6 完成：

- 采用 BSGS 策略将旋转次数从 8 次降低到 4 次
- 通过 2 次 Baby-Step 旋转和 2 次 Giant-Step 旋转达到理论下界
- BSGS 法最大误差 3.124567×10^{-7}
- 5 组随机测试全部通过
- 加速比 2.224 倍

理论分析完成：

- 证明 4 次是两级 BSGS 模型的最小值
- 证明 4 次是一般旋转-掩码-加法模型的普遍下界

7.2 实验感悟

关于环境配置：从 vcpkg 自动安装遇到网络问题，到最终手动编译 SEAL 库，这个过程让我认识到掌握从源码编译的能力是应对工程依赖问题的基本素养。

关于性能优化：BSGS 策略的核心是利用偏移的二维结构进行复用。同态加密的程序设计本质上是在“计算能力受限”的条件下重新设计算法，需要重点识别和优化高成本操作（旋转、重线性化等）。

关于近似加密：CKKS 的近似特性要求在设计应用时预留足够的精度裕量。本实验误差在 10^{-5} 到 10^{-7} 量级，远小于 10^{-4} 的阈值，参数选择合理。

关于理论指导：旋转次数下界的证明说明，工程优化不能仅靠经验，需要理论分析来指导方向。理解了“为什么是 4 次”比仅仅“实现 4 次”更有价值。